

Battery Dispatch Optimizer — Written Formulation

Day 1 Submission | Zewail City of Science and Technology

1. Overview

The battery dispatch optimizer is implemented as a **Model Predictive Control (MPC)** controller with a rolling horizon of **H = 96 timesteps (1 full day)** at 15-minute resolution. At the start of each day, the LightGBM forecaster provides a 96-step load forecast. The optimizer solves a **Linear Program (LP)** over this horizon, then executes all 96 dispatch decisions for the day before rolling to the next day.

Parameter	Value
Horizon H	96 steps (1 day = 24 hours)
Time resolution (Delta t)	0.25 h (15 minutes)
Optimization method	Linear Programming (LP) — CVXPY / PuLP
Rolling scheme	Day-ahead: solve once per day, execute all 96 steps
Forecast input	LightGBM day-ahead load forecast (96 values)
PV input	Actual PV generation (deterministic from irradiance)
Price input	Actual buy/sell prices from dataset

2. System Specifications

Parameter	Value	Notes
Solar PV capacity	9 kWp	Peak at standard test conditions
Battery usable capacity (C)	16 kWh	Total usable energy
Max charge / discharge power	8 kW	Symmetric limit
Round-trip efficiency	90%	$\sqrt{0.90}$ per direction = 0.9487
Charge efficiency (η_c)	0.9487	$\sqrt{0.9}$
Discharge efficiency (η_d)	0.9487	$\sqrt{0.9}$
Grid connection limit	6 kW	Max import and export
Initial SoC	50%	At start of optimization window
Delta t	0.25 h	15 minutes per timestep

3. Power Sign Convention

Variable	Sign	Meaning
----------	------	---------

$P_{\text{battery}}(t)$	Negative (< 0)	Battery charging
$P_{\text{battery}}(t)$	Positive (> 0)	Battery discharging
$P_{\text{grid}}(t)$	Positive (> 0)	Importing from grid (cost)
$P_{\text{grid}}(t)$	Negative (< 0)	Exporting to grid (revenue)

4. Objective Function

At each day d , minimize the total electricity bill over the 96-step horizon:

```

minimize Bill = SUM over t in {0..95} of:
[ buy_price(t) * P_grid_import(t)
- sell_price(t) * P_grid_export(t) ] * Delta_t

```

where:

```
P_grid_import(t) = max(P_grid(t), 0) [import portion, >= 0]
```

```
P_grid_export(t) = max(-P_grid(t), 0) [export portion, >= 0]
```

This is linearized for LP by introducing auxiliary variables $p_{\text{import}}(t) \geq 0$ and $p_{\text{export}}(t) \geq 0$ such that $P_{\text{grid}}(t) = p_{\text{import}}(t) - p_{\text{export}}(t)$.

5. Decision Variables

Variable	Unit	Description
$P_{\text{battery}}(t)$	kW	Battery power at timestep t (+ discharge, - charge)
$P_{\text{grid}}(t)$	kW	Grid power at timestep t (+ import, - export)
$p_{\text{import}}(t)$	kW	Auxiliary: grid import portion (≥ 0)
$p_{\text{export}}(t)$	kW	Auxiliary: grid export portion (≥ 0)
$\text{SoC}(t)$	p.u.	State of charge at timestep t (0 to 1)

6. Hard Constraints

6.1 Energy Balance (must hold at every timestep)

```
load_forecast(t) = pv(t) + P_grid(t) + P_battery(t) for all t
```

The site load must be fully served at every 15-minute interval without exception.

6.2 Grid Power Limits

```
-6 kW <= P_grid(t) <= 6 kW for all t
```

```
P_grid(t) = p_import(t) - p_export(t)
```

```
p_import(t) >= 0, p_export(t) >= 0
```

6.3 Battery Power Limits

```
-8 kW <= P_battery(t) <= 8 kW for all t
```

6.4 State of Charge Limits

0 <= SoC(t) <= 1 for all t

6.5 SoC Dynamics (Battery Model)

The SoC evolves according to charging and discharging efficiency:

For charging (P_battery < 0):

SoC(t+1) = SoC(t) + (-P_battery(t) * eta_c * Delta_t) / C

For discharging (P_battery >= 0):

SoC(t+1) = SoC(t) - (P_battery(t) / eta_d * Delta_t) / C

Linearized for LP by splitting P_battery into charge and discharge components:

P_battery(t) = p_disch(t) - p_charg(t)

p_disch(t) >= 0, p_charg(t) >= 0

SoC(t+1) = SoC(t) + (p_charg(t)*eta_c - p_disch(t)/eta_d) * Delta_t / C

6.6 Initial Condition

SoC(0) = 0.50 (50% at start of each optimization window)

7. Rolling Horizon MPC Algorithm

The controller operates causally — at each decision step it may only use information available up to that moment. The rolling horizon is set to H = 96 steps (one full day) to match the forecaster output.

- Step 1:** At start of day d (t = 0 of day), load LightGBM day-ahead forecast: load_forecast[0..95] (96 values at 15-min resolution).
- Step 2:** Load actual PV generation pv[0..95] and price signals buy_price[0..95], sell_price[0..95] for the day.
- Step 3:** Solve LP over horizon H = 96 steps with current SoC(0) as initial condition.
- Step 4:** Execute all 96 optimal dispatch decisions P_battery*(t) for the day.
- Step 5:** Update SoC for start of next day. Advance to day d+1 and repeat.

Note: A one-shot batch optimization over the full year simultaneously is NOT used. The rolling day-ahead approach reflects real controller operation.

8. Baselines for Comparison

Baseline	Description	Purpose
Baseline A (Historical)	Use recorded p_battery_kw column from dataset. Exact on-site controller operation.	Beat this = our controller outperforms existing system
Baseline B (Zero-Intelligence)	PV powers load first. Surplus exported. Deficit bought from grid. Battery not used.	Shows value of having a controllable battery at all

9. Implementation Plan (Day 2)

Task	Tool / Library	Status
LP formulation	CVXPY or PuLP	Day 2
Baseline A bill computation	Pandas + NumPy	Day 2
Baseline B bill computation	Pandas + NumPy	Day 2
Rolling horizon loop	Python (for loop over days)	Day 2
Oracle run (perfect forecast)	Same LP with actual load	Day 2
March Week 3 dispatch plot	Matplotlib	Day 2
Savings table vs both baselines	Pandas	Day 2
Extension: horizon sensitivity	H in {4, 24, 96} steps	Day 2 (if time allows)

10. Required Outputs (Day 2)

- 2025 electricity bill under Baseline A (historical operation)
- 2025 electricity bill under Baseline B (zero-intelligence)
- 2025 electricity bill under our rolling-horizon MPC controller
- Absolute savings (EUR) and percentage savings vs each baseline
- Oracle gap: re-run optimizer with actual 2025 load as input — report oracle savings, forecast-based savings, and gap in EUR and %
- Dispatch plot for March Week 3 2025: load, PV, P_battery, P_grid, and SoC